

# COSC/MATH 4340: Numerical Methods for Differential Equations

## Ordinary Differential Equations (ODEs): Boundary Value Problems (BVPs)

Class: COSC/MATH 4340

Semester: Spring 2024

Instructor: Dane Taylor

Office: Eng 4087

Email: [dane.taylor@uwyo.edu](mailto:dane.taylor@uwyo.edu)





# Introduction

Thus far we have considered initial value problems

$$\begin{cases} \frac{dy}{dt} = f(t, y) \\ y(t_0) = y_0 \end{cases}$$

for a solution  $y(t)$  over some range  $t \in [t_0, T]$ .

We now turn our focus to boundary value problems (BVPs)

## Definition

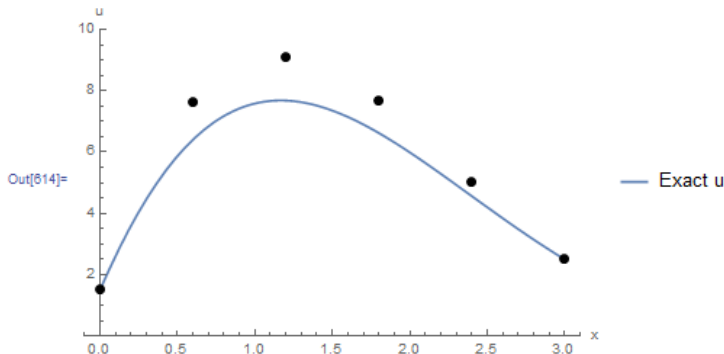
A **2nd-order boundary value problem** takes the form

$$\begin{cases} y''(t) = f(t, y) \\ y(a) = y_a \\ y(b) = y_b. \end{cases}$$

# Introduction

Constraints must be matched on the left and right:

$$y(a) = y_a \text{ and } y(b) = y_b.$$





# Existence of Solutions

Understanding the existence and uniqueness of solutions to BVPs is much more complicated than for IVPs (which just required a Lipschitz constant).

BVPs often have either 1 solution, no solutions, or infinite solutions.

(See examples)



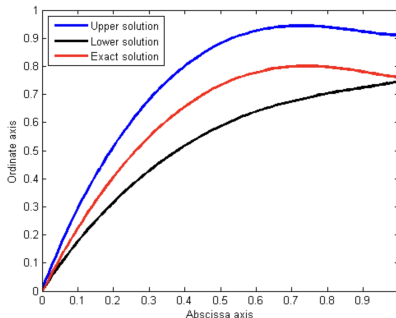
# Overview

We will study 3 methods for solving BVPs:

- ▶ **Shooting methods:** use IVP numerical methods (e.g., FE, RK, trapezoidal) with  $y_0 = y_a$  and iteratively find a solution  $\{y_i\}$  so that  $y_N$  converges toward  $y_b$ .
- ▶ **Finite difference methods:** use approximations  $y'(t_i) \approx (y_{i+1} - y_i)/h$  and construct a large sparse linear system that must be solved with a linear solver.
- ▶ **Finite element methods:** use a functional basis defined over  $t \in [a, b]$  (e.g., the set of polynomials,) and then solve for the basis elements (i.e., coefficients)

# The Shooting Method

Use an IVP solver such as forward Euler or Runge-Kutta to integrate from  $t = a$  to  $t = b$  and iteratively adjust the initial condition until the right-hand side is close to  $y_b$ .





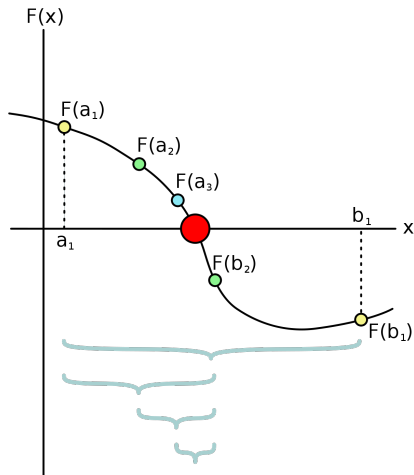
# The Shooting Method

## Outline of method

- ▶ Rewrite the 2nd-order BVP as a 2D system of equations
- ▶ Set the initial conditions  $y(a) = y_a$  and  $y'(a) = s$  for some initial choice  $s$  (which will be refined).
- ▶ Use a numerical method to solve the IVP over the range  $[a, b]$  and define  $G(s) = y_N - y_b$  as the error on the right-hand side.
- ▶ Use a root-finding algorithm (e.g., bisection method) to approximate the value  $s^*$  in which  $G(s^*) = 0$ .

(See example.)

# Recall the Bisection Root-finding Method





## Recall the Bisection Root-finding Method

### Definition

Starting with any two values  $a_0$  and  $b_0$  such that  $G(a_0)G(b_0) < 0$ , let  $c_0 = (a_0 + b_0)/2$  be the midpoint and compute  $G(c_0)$ . If  $G(a_0)G(c_0) > 0$ , define  $a_1 = c_0$  and  $b_1 = b_0$  (otherwise define  $a_1 = a_0$  and  $b_1 = c_0$ ). Then  $c_i$  converges to a root and intervals  $[a_t, b_t]$  contain the root and become smaller and smaller with  $t$ . Iterations stop after a tolerance is reached or after  $T$  iterations.

Bisection method is guaranteed to converge when  $f$  is continuous, but it is a bit slow.

The error decays as  $2^{-t}$  since the range  $[a_t, b_t]$  is halved each iteration.



## Recall Root-finding using Fixed Point Iteration

### Theorem

*Consider  $x = g(x)$ , and let  $g'(x^*)$  denote the derivative of  $g(x)$  at  $x = x^*$ . Then the sequence  $x_{t+1} = g(x_t)$  converges to a solution  $x^*$  if and only if  $|g'(x^*)| < 1$ .*

To find a root  $s^*$  of  $G(s)$ , we can define  $g(s) = s + G(s)$ . It follows that  $g(s^*) = s^*$  iff  $G(s) = 0$ .

The convergence rate is  $|e_t| \leq M^t |e_0|$ , where  $M = \max_{\xi \in D} |g'(\xi)|$  bounds the derivative's magnitude over some region  $D$  that contains  $x^*$ . This is faster than bisection method when  $M < 1/2$ .



# BVP Problems involving Systems of Equations

Example: Using the shooting method to solve a BVP problem that involves a 2D system of equations:

$$\begin{cases} y_1'(t) = f_1(t, \mathbf{y}) = y_1(t) - 3y_1(t)y_2(t) \\ y_2'(t) = f_2(t, \mathbf{y}) = -6(ty_2(t) + \ln(y_1(t))) \\ y_1(0) = 1 \\ y_2(1) = -2/3. \end{cases}$$

(See notes and code)



## Finite Difference Methods

Solve a BVP by using finite difference approximations to construct a large system in which all the approximations  $y_0, y_1, y_2, \dots, y_N$  are solved at the same time.

Example: 3 finite differences to estimate the slope at  $t_i$ :

$$y'(t) = \frac{y(t+h) - y(t)}{h} - \frac{h}{2}y''(\xi) \quad (\text{forward difference})$$

$$y'(t) = \frac{y(t) - y(t-h)}{h} + \frac{h}{2}y''(\xi) \quad (\text{backward difference})$$

$$y'(t) = \frac{y(t+h) - y(t-h)}{2h} - \frac{h^2}{6}y'''(\xi) \quad (\text{centered difference})$$

(See notes.)



## Higher-Order Finite Differences

Example: Approximating the second derivative

$$y''(t) = \frac{y(t+h) - 2y(t) + y(t-h)}{h^2} - \frac{h^2}{12}y^{(4)}(\xi)$$

(See notes.)



## Finite Difference Methods

Consider a 2nd-order boundary value problem

$$\begin{cases} y''(t) &= f(t, y) \\ y(a) &= y_a \\ y(b) &= y_b. \end{cases}$$

The finite difference method is as follows:

- ▶ Choose the number  $N$  of steps and let  $h = (b - a)/N$  denote the step size
- ▶ Define a mesh  $t_0, t_1, \dots, t_N$  so that  $t_0 = a$  and  $t_N = b$ .
- ▶ Define an approximation in which  $y_0 = y(a)$ ,  $y_N = y(b)$ , and the remaining  $y_i$  values are found by approximating  $y''(t)$  and  $f(t, y)$  at each  $t_i$  and  $y_i$  using finite differences.



## Example 1: Linear BVP

$$\begin{cases} y''(t) &= 4y \\ y(0) &= 1 \\ y(1) &= 3. \end{cases}$$

Let  $y_i$  estimate  $y(t_i)$  and set  $y_0 = 1$  and  $y_N = 3$ . We obtain the other estimates through a finite difference equation:

$$y''(t_i) \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} = 4y_i$$

or

$$y_{i+1} + (-2 - 4h^2)y_i + y_{i-1} = 0$$

(See notes and code.)



# Newton's Root-finding Method

## Definition

**Newton's method** for root finding uses the iterative process

$$x_{t+1} = x_t - \frac{f(x_t)}{f'(x_t)} \quad (\text{scalar } x) \quad (1)$$

$$\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - [J(\mathbf{x}^{(t)})]^{-1} F(\mathbf{x}^{(t)}), \quad (\text{vector } \mathbf{x}) \quad (2)$$

where  $J(\mathbf{x})$  is the **Jacobian matrix** for vector function  $F(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_n(\mathbf{x})]$  in which  $J_{ij}(\mathbf{x}) = \frac{\partial}{\partial x_j} f_i(\mathbf{x})$ .

For non-repeated roots, Newton's method converges quadratically:  
 $e_{t+1} = ce_t^2$ .

(See notes.)



## Example 2: Nonlinear BVP

$$\begin{cases} y''(t) &= y'(t) + \cos(y) \\ y(0) &= 0 \\ y(\pi) &= 1. \end{cases}$$

Let  $y_i$  estimate  $y(t_i)$  and set  $y_0 = 0$  and  $y_N = 1$ . We obtain the other estimates through a finite difference equation that uses the following finite differences

$$y''(t_i) \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2}$$

$$y'(t_i) \approx \frac{y_{i+1} - y_{i-1}}{2h}$$

(See notes and code.)



# Collocation Method

Given a 2nd-order BVP

$$\begin{cases} y''(t) &= f(t, y(t), y'(t)) \\ y(a) &= y_a \\ y(b) &= y_b, \end{cases}$$

we approximate the function as

$$y(t) = \sum_{j=0}^N c_j \phi_j(t) = \sum_{j=0}^N c_j t^j,$$

In principle, one can choose other basis functions besides polynomials  $\phi_j(t) = t^j$ .



## Collocation Method

First we define a mesh of time points,  $t_0 = a, t_1, \dots, t_{N-1}, t_N = b$ , where  $t_i = a + ih$  and  $h = (b - a)/N$ .

Then, we obtain the unknown coefficients  $c_j$  by satisfying the two boundary conditions

$$y(a) = \sum_{j=0}^N c_j a^j, \quad y(b) = \sum_{j=0}^N c_j b^j$$

and then by solving the differential equation at the interior time points

$$y''(t_i) = f(t_i, y(t_i), y'(t_i)), \quad i = 1, 2, \dots, N-1$$



# Collocation Method

These equations use that

$$y(t) = \sum_{j=0}^N c_j \phi_j(t) = \sum_{j=0}^N c_j t^j$$

$$y'(t) = \sum_{j=0}^N c_j \phi_j'(t) = \sum_{j=0}^N c_j j t^{j-1}$$

$$y''(t) = \sum_{j=0}^N c_j \phi_j''(t) = \sum_{j=0}^N c_j j(j-1) t^{j-2}$$



## Collocation Example

$$\begin{cases} y''(t) &= 4y(t) \\ y(0) &= 1 \\ y(1) &= 3. \end{cases}$$

Observe for each  $t_i$  that the differential equation implies

$$\sum_{j=0}^N c_j j(j-1)t_i^{j-2} = 4 \sum_{j=0}^N c_j t_i^j$$

(See notes and code.)



## Finite Elements Method

Similar to collocation, we numerically solve a 2nd-order BVP  $y''(t) = f(t, y(t), y'(t))$  by approximating the function as

$$y(t) = \sum_{j=0}^N c_j \phi_j(t),$$

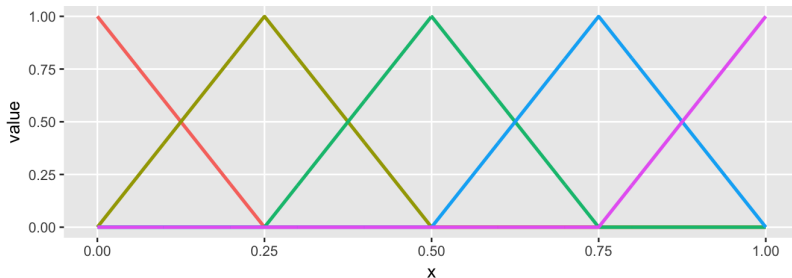
and we again must solve for the coefficients  $c_j$ .

- ▶ For collocation we assumed  $\phi_j(t) = t^j$ , whereas for **finite elements** we assume  $\phi_j(t)$  are B-splines.
- ▶ Moreover, for collocation we obtained the  $c_j$  values by evaluating the system at times  $t_i$ ; however, for the **Galerkin finite element** method the  $c_j$  values minimize the squared error of the differential equation *along the solution*.



## Piecewise Linear B-Splines

Each  $\phi_j(t)$  is a “tent” function that is zero everywhere, except for the range  $t \in [t_{j-1}, t_{j+1}]$ , over which it forms a linear tent.



Observe that  $\phi_j(t_i) = 1$  when  $i = j$  and  $\phi_j(t_i) = 0$  when  $i \neq j$ .  
Therefore if  $y(t) = \sum_{j=0}^N c_j \phi_j(t)$ , then  $y(t_i) = c_i$ .



## Piecewise Linear B-Splines

$$\phi_0(t) = \begin{cases} \frac{t_1 - t}{t_1 - t_0} & \text{for } t_0 \leq t \leq t_1 \\ 0 & \text{otherwise} \end{cases}$$

$$\phi_N(t) = \begin{cases} \frac{t - t_{N-1}}{t_N - t_{N-1}} & \text{for } t_{N-1} \leq t \leq t_N \\ 0 & \text{otherwise} \end{cases}$$

$$\phi_i(t) = \begin{cases} \frac{t - t_{i-1}}{t_i - t_{i-1}} & \text{for } t_{i-1} \leq t \leq t_i \\ \frac{t_{i+1} - t}{t_{i+1} - t_i} & \text{for } t_i \leq t \leq t_{i+1} \\ 0 & \text{otherwise} \end{cases}$$

(See code.)



## Galerkin Finite Element Method

Consider solving  $y''(t) = f(t, y, y')$  with the assumed form  $y(t) = \sum_{j=0}^N c_j \phi_j(t)$ . To minimize the squared error, the residual  $y''(t) - f(t, y, y')$  must be orthogonal to the basis functions so that

$$\int_a^b \left( y''(t) - f(t, y, y') \right) \phi_i(t) dt,$$

which is called the **weak form** of the BVP.

(See notes.)



## Galerkin Finite Element Method

This can also be simplified using integration by parts''

$$\begin{aligned}\int_a^b f(t, y, y') \phi_i(t) dt &= \int_a^b y''(t) \phi_i(t) dt \\ &= y'(b) \phi_i(b) - y'(a) \phi_i(a) - \int_a^b y'(t) \phi_i'(t) dt \\ &= 0 - \int_a^b y'(t) \phi_i'(t) dt,\end{aligned}$$

which assumes  $i$  is neither 0 or  $N$  and uses that  $\phi_i(a) = \phi_i(t_0) = 0$  and  $\phi_i(b) = \phi_i(t_N) = 0$ .



## Galerkin Finite Element Method

The coefficients  $c_j$  in  $y(t) = \sum_{j=0}^N c_j \phi_j(t)$  can thus be solved as:

$$j = 0 : \quad y(a) = \sum_{j=0}^N c_j \phi_j(a) = c_0$$

$$j = N : \quad y(b) = \sum_{j=0}^N c_j \phi_j(b) = c_N$$

$$\text{other } j : \quad \int_a^b f(t, y, y') \phi_j(t) dt + \int_a^b y'(t) \phi_j'(t) dt = 0$$



## Galerkin Finite Element Method

Importantly, these integrals can be simplified after using the following properties of piecewise-linear B-splines

$$\langle \phi_i, \phi_j \rangle = \int_a^b \phi_i(t) \phi_j(t) dt = \begin{cases} 2h/3 & \text{if } j = i \\ h/6 & \text{if } |j - i| = 1 \\ 0 & \text{otherwise} \end{cases}$$

$$\langle \phi'_i, \phi'_j \rangle = \int_a^b \phi'_i(t) \phi'_j(t) dt = \begin{cases} 2/h & \text{if } j = i \\ -1/h & \text{if } |j - i| = 1 \\ 0 & \text{otherwise} \end{cases}$$



# Galerkin Finite Element Methods

Example:

$$\begin{cases} y''(t) &= 4y(t) \\ y(0) &= 1 \\ y(1) &= 3. \end{cases}$$

Note that we consider the equations

$$\begin{aligned} 0 &= \int_0^1 \phi_i(t) f(t, y, y') dt + \int_0^1 \phi'_i(t) y'(t) dt \\ &= \int_0^1 \left[ \phi_i(t) 4 \sum_{j=0}^N c_j \phi_j(t) + \phi'_i(t) \sum_{j=0}^N c_j \phi'_j(t) \right] dt \end{aligned}$$

(See notes and code.)